



1.- CAPÍTULO II: Programación Orientada a Objetos

Conceptos Introdutorios

Marzo 2026 - Agosto 2026

Ciencia de Datos & Inteligencia Artificial

1. Introducción

2. Paradigmas de Programación

2.1 Definición

2.2 Historia

3. Tipos de Programación

3.1 Programación Imperativa

3.2 Programación Estructurada

3.3 Programación Procedimental

3.4 Programación Orientada a Objetos (POO)

3.5 Programación Funcional

3.6 Programación Lógica

3.7 Programación Concurrente

3.8 Programación Orientada a Eventos

3.9 Programación Orientada a Aspectos

4. Lenguajes de Programación

4.1 Reflexión

Si pudieras preguntarles por qué lo hacían, su respuesta sería:



- "Porque así es como siempre se han hecho las cosas aquí."
- Antes de seguir lo que todos hacen, pregúntate por qué. Porque entender es mejor que repetir.
- Hoy seguimos repitiendo reglas, tradiciones y prejuicios que quizás algunos no recuerdan de dónde vienen.

1. Introducción

2. Paradigmas de Programación

2.1 Definición

2.2 Historia

3. Tipos de Programación

3.1 Programación Imperativa

3.2 Programación Estructurada

3.3 Programación Procedimental

3.4 Programación Orientada a Objetos (POO)

3.5 Programación Funcional

3.6 Programación Lógica

3.7 Programación Concurrente

3.8 Programación Orientada a Eventos

3.9 Programación Orientada a Aspectos

4. Lenguajes de Programación

4.1 Reflexión

1. Introducción

2. Paradigmas de Programación

2.1 Definición

2.2 Historia

3. Tipos de Programación

3.1 Programación Imperativa

3.2 Programación Estructurada

3.3 Programación Procedimental

3.4 Programación Orientada a Objetos (POO)

3.5 Programación Funcional

3.6 Programación Lógica

3.7 Programación Concurrente

3.8 Programación Orientada a Eventos

3.9 Programación Orientada a Aspectos

4. Lenguajes de Programación

4.1 Reflexión

- Paradigma de programación es una propuesta tecnológica adoptada por una comunidad de programadores cuyo núcleo central es incuestionable en cuanto a que unívocamente trata de resolver uno o varios problemas claramente delimitados.
- La resolución de estos problemas debe suponer consecuentemente un avance significativo en al menos un parámetro que afecte a la ingeniería de Software.
- Los lenguajes de programación suelen implementar, a menudo de forma parcial, varios paradigmas

Desde los inicios de la computación, los programadores han buscado formas más eficientes y organizadas de desarrollar software.

A medida que los sistemas crecieron en complejidad, surgieron diferentes modelos conceptuales para resolver problemas computacionales.

Un paradigma de programación define:

- la manera en que se estructura un programa,
- cómo se representan los datos,
- cómo se ejecutan las instrucciones,
- y cómo interactúan los componentes del sistema.

En otras palabras, un paradigma es una filosofía de desarrollo de software.

Definición

Un paradigma de programación es un conjunto de conceptos, técnicas, metodologías y reglas que orientan la construcción de programas.

Cada paradigma propone:

- Una visión diferente del problema
- Una forma distinta de modelar soluciones,
- Mecanismos específicos para controlar el flujo de ejecución.

1. Introducción

2. Paradigmas de Programación

2.1 Definición

2.2 Historia

3. Tipos de Programación

3.1 Programación Imperativa

3.2 Programación Estructurada

3.3 Programación Procedimental

3.4 Programación Orientada a Objetos (POO)

3.5 Programación Funcional

3.6 Programación Lógica

3.7 Programación Concurrente

3.8 Programación Orientada a Eventos

3.9 Programación Orientada a Aspectos

4. Lenguajes de Programación

4.1 Reflexión

La evolución de los paradigmas está relacionada con la evolución del hardware y las necesidades del software.

Programación de bajo nivel

- Lenguaje máquina
- Ensamblador

La programación era difícil, dependiente del hardware y propensa a errores.

Programación Imperativa y Estructurada

- Surgen lenguajes como: FORTRAN, COBOL, ALGOL, Pascal
- Se introducen: estructuras de control, modularidad y reducción del uso de saltos.

Programación Orientada a Objetos

- Surgen lenguajes como: Smalltalk, C++, Objective C

La complejidad del software obliga a representar sistemas mediante objetos.

Década de 1990: Programación Funcional y Distribuida

Incremento del procesamiento paralelo y sistemas distribuidos.

Actualidad: Multiparadigma

Lenguajes modernos como Python combinan múltiples paradigmas: imperativo, orientado a objetos, funcional, concurrente.

Los paradigmas buscan:

- Simplificar el desarrollo
- Mejorar legibilidad
- Facilitar mantenimiento
- Aumentar reutilización
- Optimizar rendimiento

2.3. Clasificación General

Los paradigmas de programación pueden clasificarse en dos grandes grupos:

Paradigmas Imperativos

Describen cómo resolver un problema mediante instrucciones secuenciales. El programa modifica estados mediante: variables, asignaciones, estructuras de control.

Incluyen: programación imperativa, estructurada, procedimental, orientada a objetos.

Paradigmas Declarativos

Describen qué se desea obtener, sin especificar exactamente cómo hacerlo. Incluyen: programación funcional, programación lógica.

1. Introducción

2. Paradigmas de Programación

2.1 Definición

2.2 Historia

3. Tipos de Programación

3.1 Programación Imperativa

3.2 Programación Estructurada

3.3 Programación Procedimental

3.4 Programación Orientada a Objetos (POO)

3.5 Programación Funcional

3.6 Programación Lógica

3.7 Programación Concurrente

3.8 Programación Orientada a Eventos

3.9 Programación Orientada a Aspectos

4. Lenguajes de Programación

4.1 Reflexión

1. Introducción

2. Paradigmas de Programación

2.1 Definición

2.2 Historia

3. Tipos de Programación

3.1 Programación Imperativa

3.2 Programación Estructurada

3.3 Programación Procedimental

3.4 Programación Orientada a Objetos (POO)

3.5 Programación Funcional

3.6 Programación Lógica

3.7 Programación Concurrente

3.8 Programación Orientada a Eventos

3.9 Programación Orientada a Aspectos

4. Lenguajes de Programación

4.1 Reflexión

La programación imperativa se basa en instrucciones que modifican el estado del programa. El procesador ejecuta órdenes paso a paso.

Características:

- Variables mutables: El contenido de las variables cambia durante la ejecución.
- Secuencia de instrucciones: Las instrucciones se ejecutan en orden.
- Se asemeja a la arquitectura de Von Neumann: memoria, procesador, instrucciones secuenciales.

Ventajas:

- Simplicidad conceptual: Fácil de aprender.
- Cercanía al hardware: Muy eficiente computacionalmente.

Desventajas:

- Dependencia del estado: Los cambios constantes dificultan depuración.
- Baja escalabilidad: En proyectos grandes se vuelve complejo.

1. Introducción

2. Paradigmas de Programación

2.1 Definición

2.2 Historia

3. Tipos de Programación

3.1 Programación Imperativa

3.2 Programación Estructurada

3.3 Programación Procedimental

3.4 Programación Orientada a Objetos (POO)

3.5 Programación Funcional

3.6 Programación Lógica

3.7 Programación Concurrente

3.8 Programación Orientada a Eventos

3.9 Programación Orientada a Aspectos

4. Lenguajes de Programación

4.1 Reflexión

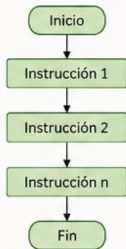
La programación estructurada establece que cualquier algoritmo puede construirse usando únicamente: Secuencia, Selección e Iteración

Modularidad:

- La programación estructurada introduce módulos o bloques lógicos, lo que mejora la organización, legibilidad y reutilización.

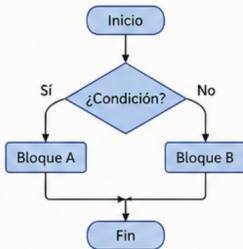
3.2. Programación Estructurada

1. SECUENCIA



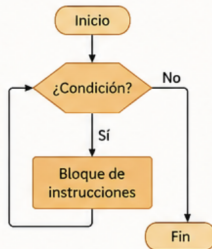
Las instrucciones se ejecutan en orden, una tras otra.

2. SELECCIÓN



Permite tomar decisiones ejecutando diferentes bloques según una condición.

3. ITERACIÓN



Permite repetir un bloque de instrucciones mientras se cumpla una condición.

Ventajas:

- Legibilidad: El código es más claro.
- Mantenimiento: Los errores son más fáciles de localizar.
- Fiabilidad: Reduce errores lógicos.

Desventajas.- Aunque mejora la organización:

- Los datos siguen separados de las funciones,
- La reutilización sigue siendo limitada.

1. Introducción

2. Paradigmas de Programación

2.1 Definición

2.2 Historia

3. Tipos de Programación

3.1 Programación Imperativa

3.2 Programación Estructurada

3.3 Programación Procedimental

3.4 Programación Orientada a Objetos (POO)

3.5 Programación Funcional

3.6 Programación Lógica

3.7 Programación Concurrente

3.8 Programación Orientada a Eventos

3.9 Programación Orientada a Aspectos

4. Lenguajes de Programación

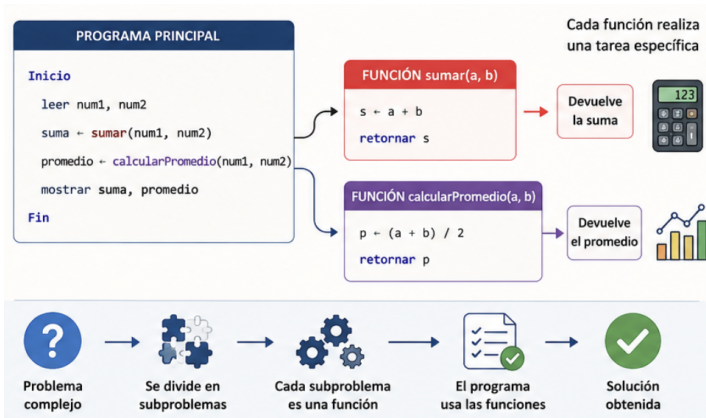
4.1 Reflexión

La programación procedimental organiza el código mediante procedimientos o funciones. Cada función realiza una tarea específica.

Característica:

- Un problema complejo se divide en: subproblemas y funciones independientes.
- Modularidad: Cada función es un módulo.
- Reutilización parcial: Las funciones pueden reutilizarse.
- Separación funcional: Cada procedimiento tiene una responsabilidad específica.

3.3. Programación Procedimental



Ventajas:

- Organización: El código se divide en partes pequeñas.
- Mantenimiento: Más fácil modificar funciones individuales.

Desventajas.- Aunque mejora la organización:

- Escasa protección de datos: los datos pueden modificarse desde cualquier parte.
- Problemas de escalabilidad: grandes sistemas generan muchas dependencias.

1. Introducción

2. Paradigmas de Programación

2.1 Definición

2.2 Historia

3. Tipos de Programación

3.1 Programación Imperativa

3.2 Programación Estructurada

3.3 Programación Procedimental

3.4 Programación Orientada a Objetos (POO)

3.5 Programación Funcional

3.6 Programación Lógica

3.7 Programación Concurrente

3.8 Programación Orientada a Eventos

3.9 Programación Orientada a Aspectos

4. Lenguajes de Programación

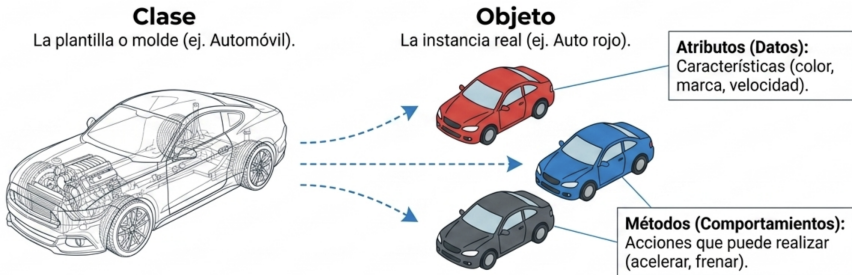
4.1 Reflexión

La POO surge para modelar entidades del mundo real.

Combina datos y comportamientos en una sola entidad llamada objeto.

Característica:

- Define los conceptos de Clase, Objeto, Atributo y Método
- Clase: Automóvil
- Objetos: Auto rojo, Auto azul, Camioneta
- Atributos: Representan características (color, marca, velocidad)
- Métodos: Representan acciones (acelerar, frenar)



Ventajas:

- Reutilización: Gracias a la herencia.
- Escalabilidad: Ideal para sistemas grandes.
- Mantenimiento: Los cambios afectan módulos específicos.
- Modelado realista: Representa entidades reales.

Desventajas

- Complejidad inicial: Requiere diseño previo.
- Mayor consumo de recursos: Los objetos utilizan memoria adicional.

1. Introducción

2. Paradigmas de Programación

2.1 Definición

2.2 Historia

3. Tipos de Programación

3.1 Programación Imperativa

3.2 Programación Estructurada

3.3 Programación Procedimental

3.4 Programación Orientada a Objetos (POO)

3.5 Programación Funcional

3.6 Programación Lógica

3.7 Programación Concurrente

3.8 Programación Orientada a Eventos

3.9 Programación Orientada a Aspectos

4. Lenguajes de Programación

4.1 Reflexión

La programación funcional se basa en funciones matemáticas puras.

Evita: cambios de estado y variables mutables.

Característica:

- Función pura: siempre produce el mismo resultado, no modifica variables externas.
- Funciones de orden superior: que reciben funciones, o retornan funciones.

EJEMPLO EN UN LENGUAJE FUNCIONAL (HASKELL)

```
1  -- Función pura
2  cuadrado x = x * x
3
4  -- Función de orden superior
5  aplicar f x = f x
6
7  -- Usando aplicar con la función cuadrado
8  resultado = aplicar cuadrado 5
9  -- resultado = 25
```

- ✓ No hay variables mutables.
- ✓ No hay cambios de estado.
- ✓ Todo se basa en la evaluación de expresiones.

Ventajas:

- Menos errores: Al evitar estados mutables.
- Paralelismo

Desventajas

- Complejidad conceptual: Menos intuitiva

1. Introducción

2. Paradigmas de Programación

2.1 Definición

2.2 Historia

3. Tipos de Programación

3.1 Programación Imperativa

3.2 Programación Estructurada

3.3 Programación Procedimental

3.4 Programación Orientada a Objetos (POO)

3.5 Programación Funcional

3.6 Programación Lógica

3.7 Programación Concurrente

3.8 Programación Orientada a Eventos

3.9 Programación Orientada a Aspectos

4. Lenguajes de Programación

4.1 Reflexión

Basada en lógica matemática. El programador define hechos, reglas y relaciones.

Característica:

- Motor de inferencia: El sistema deduce respuestas automáticamente.
- Aplicaciones: inteligencia artificial, sistemas expertos, procesamiento de lenguaje natural.

UTN
IBARRA - ECUADOR


```
graph TD; Juan[Juan] -- padre --> Pedro[Pedro]; Pedro -- padre --> Carlos[Carlos]; Pedro -- padre --> Sofia[Sofía];
```


Significado:

Si una persona es padre de otra (Y)
y esa otra persona (Y) es padre de alguien (Z),
entonces la primera persona (X) es abuelo de (Z).


```
Z = carlos ;
Z = sofia.
```



El sistema busca a quiénes es padre alguien que es hijo de Juan.
Como Pedro es hijo de Juan, y Pedro es padre de Carlos y Sofía, el sistema **deduce** que **Juan** es abuelo de **Carlos** y de **Sofía**.

1. Introducción

2. Paradigmas de Programación

2.1 Definición

2.2 Historia

3. Tipos de Programación

3.1 Programación Imperativa

3.2 Programación Estructurada

3.3 Programación Procedimental

3.4 Programación Orientada a Objetos (POO)

3.5 Programación Funcional

3.6 Programación Lógica

3.7 Programación Concurrente

3.8 Programación Orientada a Eventos

3.9 Programación Orientada a Aspectos

4. Lenguajes de Programación

4.1 Reflexión

Permite ejecutar múltiples tareas simultáneamente. Mejorar rendimiento, aprovechar múltiples núcleos y manejar procesos paralelos.

Característica:

- Multihilo: Múltiples hilos dentro de un proceso.
- Multiproceso: Múltiples procesos independientes.

3.7. Programación Concurrente

MULTIHILO (HILOS DENTRO DE UN PROCESO)

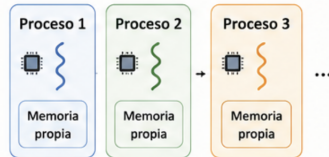
Un proceso



- Comparten la misma memoria del proceso
- Comunicación más rápida
- Menor consumo de recursos

```
// Ejemplo de creación de hilos (pseudocódigo)
crearHilo(tarea1)
crearHilo(tarea2)
crearHilo(tarea3)
```

MULTIPROCESO (MÚLTIPLES PROCESOS)



- Cada proceso tiene su propia memoria
- Aislamiento entre procesos (más seguro)
- Mayor consumo de recursos

```
// Ejemplo de creación de procesos (pseudocódigo)
crearProceso(programa1)
crearProceso(programa2)
crearProceso(programa3)
```

1. Introducción

2. Paradigmas de Programación

2.1 Definición

2.2 Historia

3. Tipos de Programación

3.1 Programación Imperativa

3.2 Programación Estructurada

3.3 Programación Procedimental

3.4 Programación Orientada a Objetos (POO)

3.5 Programación Funcional

3.6 Programación Lógica

3.7 Programación Concurrente

3.8 Programación Orientada a Eventos

3.9 Programación Orientada a Aspectos

4. Lenguajes de Programación

4.1 Reflexión

La estructura como la ejecución de los programas van determinados por los sucesos que ocurran en el sistema, definidos por el usuario o que ellos mismos provoquen.

Característica:

- La interfaz gráfica de usuario es importante, se encarga de indicar el que hacer.
- Desventaja: La mayoría de herramientas de creación de interfaces gráficas de usuario se distribuyen como frameworks (limita la programación y ejecución).
- Ejm: Visual Basic, Java script, Visual C++.

3.8. Programación Orientada a Eventos

A screenshot of a Java Swing window titled "Fecha en letras". The window contains three text input fields for date components: "Día:" with the value "16", "Mes:" with the value "4", and "Año:" with the value "1991". To the right of these fields are three buttons: "Aceptar", "Limpiar", and "Salir". Below the input fields is a label "En letras:" followed by a text area displaying the formatted date "16 de Abril de 1991".

1. Introducción

2. Paradigmas de Programación

2.1 Definición

2.2 Historia

3. Tipos de Programación

3.1 Programación Imperativa

3.2 Programación Estructurada

3.3 Programación Procedimental

3.4 Programación Orientada a Objetos (POO)

3.5 Programación Funcional

3.6 Programación Lógica

3.7 Programación Concurrente

3.8 Programación Orientada a Eventos

3.9 Programación Orientada a Aspectos

4. Lenguajes de Programación

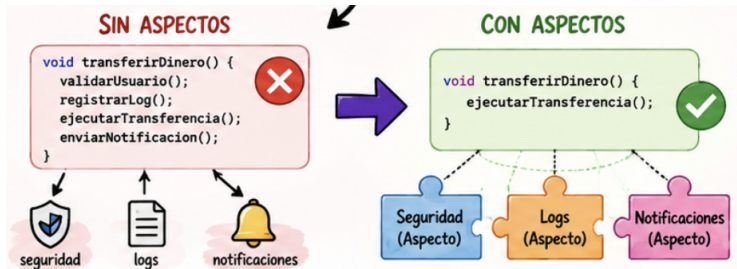
4.1 Reflexión

Aspecto que pretende dar una determinada funcionalidad al sistema; separando esta funcionalidad del sistemas, con el fin de aumentar la modularidad.

Característica:

- El código es más limpio, menos duplicado, más fácil de entender y de mantener.
- Mayor re utilización, los aspectos tienen mayores probabilidades de ser reutilizados en otros sistemas con requerimientos similares.
- Ejm: AspectJ

3.9. Programación Orientada a Aspectos



1. Introducción

2. Paradigmas de Programación

2.1 Definición

2.2 Historia

3. Tipos de Programación

3.1 Programación Imperativa

3.2 Programación Estructurada

3.3 Programación Procedimental

3.4 Programación Orientada a Objetos (POO)

3.5 Programación Funcional

3.6 Programación Lógica

3.7 Programación Concurrente

3.8 Programación Orientada a Eventos

3.9 Programación Orientada a Aspectos

4. Lenguajes de Programación

4.1 Reflexión

1. Introducción

2. Paradigmas de Programación

2.1 Definición

2.2 Historia

3. Tipos de Programación

3.1 Programación Imperativa

3.2 Programación Estructurada

3.3 Programación Procedimental

3.4 Programación Orientada a Objetos (POO)

3.5 Programación Funcional

3.6 Programación Lógica

3.7 Programación Concurrente

3.8 Programación Orientada a Eventos

3.9 Programación Orientada a Aspectos

4. Lenguajes de Programación

4.1 Reflexión

- Lisp(Programación funcional)
- Prolog(Programación Lógica)
- Scala: Imperativo, orientado a objetos, funcional, genérico y concurrente
- Erlang: Funcional, concurrente y distribuido
- Perl: Imperativo, orientado a objetos y funcional
- PHP: Imperativo, orientado a objetos, funcional y reflexivo
- JavaScript: Imperativo, orientado a objetos (prototipos) y funcional
- Java: Imperativo, orientado a objetos, reflexivo y genérico
- Python y Ruby: Imperativo, orientado a objetos, reflexivo y funcional
- C++: Imperativo, orientado a objetos, funcional y genérico
- C#: Imperativo, orientado a objetos, funcional (lambda), reflexivo y genérico

1. Introducción

2. Paradigmas de Programación

2.1 Definición

2.2 Historia

3. Tipos de Programación

3.1 Programación Imperativa

3.2 Programación Estructurada

3.3 Programación Procedimental

3.4 Programación Orientada a Objetos (POO)

3.5 Programación Funcional

3.6 Programación Lógica

3.7 Programación Concurrente

3.8 Programación Orientada a Eventos

3.9 Programación Orientada a Aspectos

4. Lenguajes de Programación

4.1 Reflexión

Los paradigmas representan distintas formas de resolver problemas computacionales.

No existe un paradigma universalmente mejor:

- cada uno tiene fortalezas
- limitaciones,
- y aplicaciones específicas.

La tendencia moderna es utilizar enfoques multiparadigma para aprovechar las ventajas de cada modelo.

4.2. Refrexión



Lo que el cliente dijo
que necesitaba



Cómo el fabricante
entendió lo que quería



Lo que el ingeniero de
producto diseñó



Cómo entendió los
requisitos el equipo



Lo que entendió el
consultor de negocio



Cómo fue
documentado



Lo que el cliente recibió
en casa



Lo que se cobró al
cliente



Cómo se diseñó el
soporte y atención



Lo que el cliente
realmente necesitaba